

LLM Security Evaluation Service

Security Developer Home Assignment

Design and implementation of a containerized RESTful backend service for ingesting LLM security evaluation results, comparing model versions, and analyzing test stability across evaluation runs.

Tech Stack

Python 3.12 · FastAPI · PostgreSQL · SQLAlchemy · Pydantic · Pytest · Docker

Author: Eden Amar

1. Project Overview

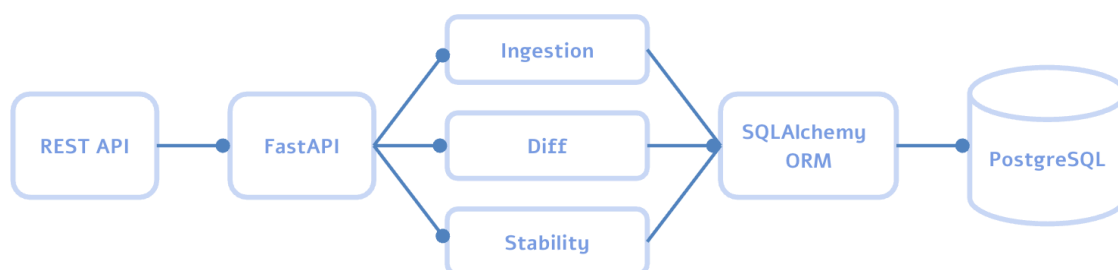
LLM safety and security behavior may change after fine-tuning, model updates, or modifications to system prompts. This project provides a backend service for storing security evaluation results and identifying behavioral changes between model versions.

The service supports three main workflows:

- Ingestion and normalized storage of evaluation runs and their findings.
- Differential analysis between two model evaluation runs.
- Stability analysis for detecting flaky or non-deterministic security tests across multiple runs.

The implementation focuses on data integrity, deterministic comparison logic, efficient cross-run analysis, and clear separation between API and business logic.

2. System Architecture



FastAPI exposes a thin REST API layer, while the main business logic is implemented in dedicated services. SQLAlchemy manages persistence and relational mapping, and PostgreSQL provides the underlying storage and integrity guarantees.

The complete environment is containerized using Docker Compose, allowing the reviewer to start the application and its dependencies with a single command.

3. Architecture Decision: PostgreSQL vs. MongoDB

Context

The incoming evaluation data is naturally represented as nested JSON. This initially made MongoDB a reasonable candidate, as the original payload could be stored directly as a document.

Decision

PostgreSQL was selected as the persistence layer.

Rationale

Although the input format is document-shaped, the underlying domain is strongly relational. A Run contains many Findings, while the same immutable TestCase can appear across many different Runs.

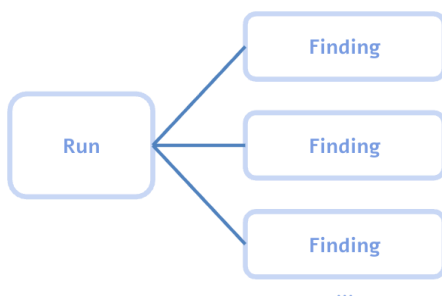
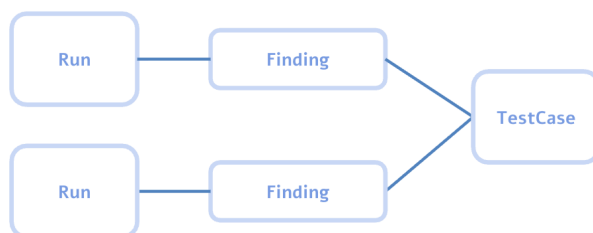
The assignment also places explicit emphasis on normalization, idempotency, integrity, and cross-run comparison. PostgreSQL provides a natural fit for these requirements through foreign keys, unique constraints, transactions, and efficient relational queries.

Conclusion: The relational characteristics of the domain were considered more important than preserving the shape of the incoming JSON.

4. Normalized Data Model

Run 1 -> N Findings

Findings N -> 1 TestCase



Run

Represents one model evaluation execution and stores its metadata and precomputed summary metrics.

TestCase

Stores the immutable definition of a security test, including `test_case_id`, `category`, `sub_category`, and `prompt`.

Finding

Represents the result of executing a specific TestCase in a specific Run. It contains the actual model output, severity, risk score, and pass/fail status.

Why normalize TestCase?

If the same TestCase participates in 100 evaluation runs, its immutable definition is stored once rather than duplicated 100 times. Findings reference that shared definition.

5. Ingestion Flow



Validation and Normalization

Incoming requests are validated before persistence. Existing TestCases are reused by their immutable `test_case_id`; new definitions are created only when necessary.

If an existing `test_case_id` is submitted with conflicting immutable fields, the request is rejected rather than silently modifying the existing definition.

Real-Time Aggregation

Summary metrics are calculated during ingestion and persisted directly on the Run record. This avoids rescanning every Finding whenever run-level information is requested.

Persisted metrics include:

- total findings
- severity counts
- average risk score

Idempotency

`run_id` acts as the external immutable identifier of an evaluation run. Re-uploading an existing `run_id` is rejected with HTTP 409 `Conflict`, preventing duplicate Findings and corrupted summary statistics.

Transaction Integrity

Run ingestion is performed as a single database transaction. A Run, its Findings, TestCases, and calculated summary are either persisted together or rolled back together.

This prevents partially ingested evaluation runs from remaining in the database after an error.

6. Differential Analysis

The differential analysis endpoint compares a **base Run** with a **head Run**. Findings are matched using the immutable `test_case_id`, allowing the service to identify how the behavior of each security test changed between model versions.

```
GET /diff?base_run_id=run_001&head_run_id=run_002
```

Classification Rules

base	head	result
Passed	Failed	New Issue
Absent	Failed	New Issue
Failed	Passed	Solved Issue
Failed / High	Failed / Critical	Worsened
Failed / Critical	Failed / High	Improved
Failed / same severity	Lower risk score	Worsened
Failed / same severity	Higher risk score	Improved
No meaningful change	No meaningful change	Unchanged

Metric Conflict Decision

A design decision was required for cases where severity and risk score move in opposite directions.

The implementation gives **severity precedence over risk score**. For example, a change from **High** to **Critical** is classified as Worsened even if the risk score improves.

This provides deterministic classification and treats severity as the primary security signal.

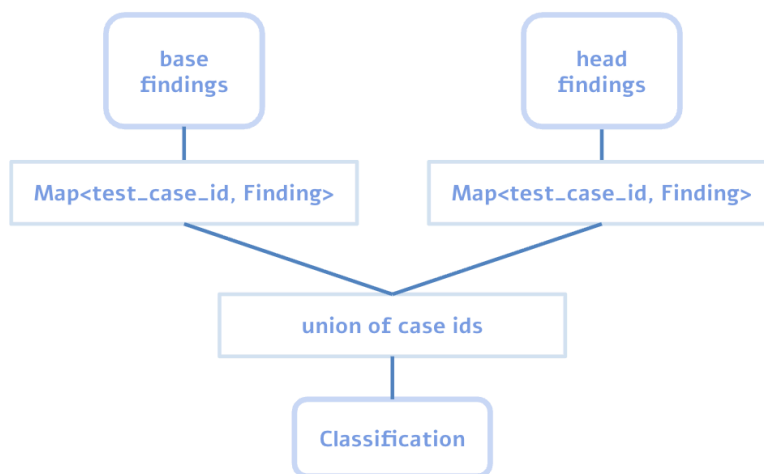
Risk Score Assumption

The assignment explicitly defines a lower risk score as degradation and a higher risk score as improvement. The implementation follows this convention, even though other risk-scoring systems may define the direction differently.

7. Comparison Algorithm

A naive implementation could compare every Finding in the base Run with every Finding in the head Run, resulting in quadratic behavior.

Instead, Findings from both Runs are indexed in memory by their immutable `test_case_id`.



*Each test ID is processed once.

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

where **n** is the number of Findings in the base Run and **m** is the number of Findings in the head Run.

Missing Tests

A TestCase that exists in the base Run but is absent from the head Run is not automatically considered solved, because absence does not imply that the test passed.

Similarly, a newly introduced TestCase that passes in the head Run is not considered a new security issue. These cases are represented separately as informational states.

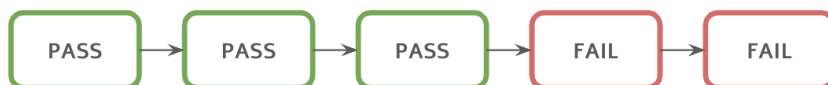
8. Stability Analysis

LLM evaluations may be non-deterministic. The same prompt can pass in one evaluation and fail in another, making it useful to distinguish stable behavior from frequently changing behavior.

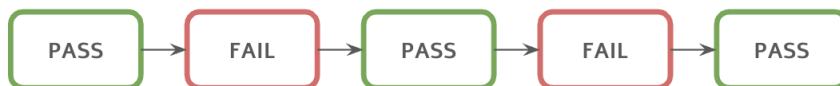
Design Question

One design question was whether flakiness should be calculated from the percentage of passes/failures or from behavioral changes between consecutive runs.

Sequence A



Sequence B



Both sequences contain exactly three passes and two failures. A percentage-based metric would therefore treat them equally.

However, Sequence B changes status on every evaluation and is clearly less stable.

Decision: Transition-Based Stability

Stability is therefore calculated from the number of status transitions between consecutive observations.

$$Stability = 100 \times \left(1 - \frac{Transitions}{Observations - 1} \right)$$

Example



5 observations | 4 possible transitions | 2 actual transitions

$$Stability = 100 \times \left(1 - \frac{2}{4} \right) = 50\%$$

100% Stability: the status never changes.

0% Stability: the status changes at every opportunity.

Stability measures consistency, not security quality. A TestCase that consistently fails can still have a Stability Score of 100%.

Latest-N Analysis

The endpoint evaluates only the latest N observations. Ordering and limiting are performed by the database rather than loading the complete history into application memory.

9. API Overview

Method	Endpoint	Purpose
GET	/health	Service health check
POST	/runs	Ingest an evaluation run
GET	/runs/{run_id}	Retrieve a run summary
GET	/diff	Compare two evaluation runs
GET	/tests/{test_case_id}/stability	Analyze test stability

Example - Diff

```
GET /diff?base_run_id=run_001&head_run_id=run_002
```

Example - Stability

```
GET /tests/JB-001/stability?n=5
```

10. Testing

Automated tests were written around business rules and failure scenarios rather than only testing successful API requests.

Database & Integrity

- relationships and foreign keys
- uniqueness constraints
- risk score boundaries
- duplicate Findings

Ingestion

- valid ingestion
- TestCase reuse
- summary calculation
- duplicate run handling
- conflicting immutable definitions
- full transaction rollback

Differential Analysis

- New / Solved / Worsened / Improved classifications
- unchanged behavior
- missing TestCases
- severity precedence
- API error cases

Stability Analysis

- always passed / always failed
- fully alternating behavior
- partially stable sequences
- insufficient history
- latest-N selection
- chronological ordering

```
workspace $ docker compose run --rm api pytest -q
[+] Creating 1/1
  ✓ Container workspace-db-1 Running
.....
81 passed in 0.96s
```

Manual verification was additionally performed through the automatically generated Swagger/OpenAPI interface.

11. Key Design Decisions

PostgreSQL over MongoDB

Relational integrity and normalization were prioritized over directly storing the incoming document structure.

Precomputed Aggregations

Run summaries are calculated during ingestion, trading a small amount of write-time work for simpler and more efficient summary reads.

Transactional Ingestion

Prevents partially persisted evaluation runs.

Severity Precedence

Provides deterministic behavior when differential metrics disagree.

Transition-Based Stability

Measures actual behavioral consistency instead of conflating stability with pass/fail percentage.

12. Production Considerations

Beyond the scope of this assignment, I would extend the service with a **dedicated UI for exploring evaluation runs**, comparing model versions, and investigating individual security findings. For larger datasets, I would introduce **pagination** and filtering capabilities across the API. I would also add structured application and **audit logging**, with **logs** shipped to **Elasticsearch** and visualized through **Kibana** dashboards to support **monitoring**, **debugging**, and **operational analysis**. Additional production considerations would include **authentication and authorization**, managed **database migrations**, observability and metrics, rate limiting, and asynchronous processing for large evaluation runs.